STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: TEST_Q002_EMAIL
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: Software Testing Debugging Concepts using AI	STATUS: In Progress

Practice Worksheet – Q002 – Email Domain Validation

Construct robust regular expression validation suites, equivalence partitioning test matrices, security boundary injections, and parameterized JUnit tests for institutional email verification.

KEY LEARNING OBJECTIVES:

- Engineer regular expression patterns matching institutional email formats (@abes.ac.in).
- Formulate Equivalence Partitioning tables covering valid, public, and malformed emails.
- Test email validation against malicious injection attacks (SQLi, XSS, and Header Injection).
- Implement data-driven parameterized tests utilizing JUnit 5 @ValueSource and @CsvSource.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Analyze institutional email format specifications and domain naming conventions.
Step 2: Formulate regular expressions validating alphanumeric usernames, dots, and institutional domains.
Step 3: Design test matrices segregating accepted vs rejected address patterns.
Step 4: Implement automated test methods asserting boolean outputs and security boundary behavior.

EXERCISE 2.1: REGEX PATTERN ENGINEERING FOR INSTITUTIONAL DOMAINS

Problem Statement: Construct a strict regex pattern for validating college email addresses ending in `@abes.ac.in`. The username must accept alphanumeric characters, dots, and underscores (length 3-30), but must not start or end with a dot. Document the regex tokens and boundary anchors.

Reference Baseline / Scenario / Code:

```
^[a-zA-Z0-9]+(?:\.[a-zA-Z0-9]+)*@abes\.ac\.in$
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Regex Pattern Definition: _____

[2] Token Explanation & Anchors: _____

[3] Username Character Set Rules: _____

EXERCISE 2.2: EQUIVALENCE PARTITIONING TEST MATRIX

Problem Statement: Create an Equivalence Partitioning table for `EmailValidator.isValid(email)`. Define 4 distinct partitions: (1) Valid Institutional, (2) Unauthorized Public Domain (@gmail.com, @yahoo.com), (3) Syntactically Malformed (missing @, space, double dot), and (4) Invalid TLDs (@abes.com, @abes.org).

Reference Baseline / Scenario / Code:

```
Partition: Unauthorized Public Domain | Test: student@gmail.com | Expected: false
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Partition Classification Table: _____

[2] Sample Test Input Payloads: _____

[3] Expected Boolean Output: _____

EXERCISE 2.3: SECURITY & SANITIZATION BOUNDARY TESTING

Problem Statement: Design test cases to ensure the email validator resists adversarial inputs. Construct test payloads simulating: (1) SQL Injection (`admin' OR '1'='1'--@abes.ac.in`), (2) Cross-Site Scripting (`<script>alert(1)</script>@abes.ac.in`), and (3) Carriage Return Header Injection (`user@abes.ac.in\nBcc:victim@test.com`).

Reference Baseline / Scenario / Code:

```
@Test void testSqlInjectionInEmail() { assertFalse(EmailValidator.isValid("admin'--@abes.ac.in")); }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Adversarial Payloads Specification: _____

[2] Sanitization & Assertion Logic: _____

[3] Security Vulnerability Safeguards: _____

EXERCISE 2.4: PARAMETERIZED JUNIT 5 TEST SUITE IMPLEMENTATION

Problem Statement: Write a JUnit 5 `@ParameterizedTest` with `@ValueSource` containing 8 diverse email variations (4 valid, 4 invalid). Verify that the test suite executes deterministically with descriptive test display names.

Reference Baseline / Scenario / Code:

```
@ParameterizedTest(name = "Testing email: {0}") @ValueSource(strings = {"aman.24j@abes.ac.in", ...})
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Parameterized Test Annotations: _____

[2] ValueSource Test Strings Array: _____

[3] Assertion Verification Code: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Regex Precision:** Pattern strictly restricts domain to @abes.ac.in without regex leaks.
- [ ] **Partition Completeness:** Covers valid, invalid domain, malformed syntax, and length extremes.
- [ ] **Security Resilience:** Blocks SQLi, XSS, and CRLF injection attempts.
- [ ] **Test Automation:** Parameterized tests run cleanly with zero hardcoded duplication.
- [ ] **Performance:** Regex evaluation completes in < 5ms without catastrophic backtracking.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is catastrophic regex backtracking a major security threat in client-submitted email fields?*
- *How does equivalence partitioning prevent testers from writing hundreds of redundant test cases?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) [ ] NEEDS REVISION (Resubmit Corrected Work) **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Edge Cases